HAI_LANGUAGE

MATLAB implementation of a hierarchical active inference model of language, reading, and eye movements.

Reference paper

This repository accompanies:

> Francesco Donnarumma, Mirco Frosolone, and Giovanni Pezzulo (2025). *Integrating large language models and active inference to understand eye movements in reading and dyslexia*. Physics of Life Reviews, 55, 61–78.

- Published article
- Open arXiv version

Overview

The model represents language with nested generative models. Each level jointly infers linguistic content and its location, while higher levels provide contextual predictions to lower levels. Active information gathering selects informative letter locations and produces sequences of simulated saccades.

A typical three-level hierarchy is:

Level	Content state	Location state	Observation
1	Syllable	Letter position	Letter and position
2	Word	Syllable position	Syllable and position
3	Sentence	Word position	Word and position

The same construction can be reduced to two levels or extended to additional levels and contextual classes. The paper uses it to study known and novel word recognition, context-sensitive reading, eye movements, and altered prior precision as a computational account of dyslexic reading.

Repository structure

- `HAI/`: construction, initialization, and execution of hierarchical active inference models.
- `VB/`: variational-Bayes update routines used by the model.
- `TREE/` and `TRACE/`: hierarchy traversal, inspection, and execution traces.
- `DICTIONARY/`: dictionary builders and the dictionaries used by the examples.
- `BERT/`: MATLAB integration with transformer-generated sentence predictions.
- `chatGPT/`: optional legacy OpenAI sentence-generation examples.
- `MAIN/`: demonstration, paper-oriented, and novel-word scripts.
- `Simulation/`: the eight simulations used to generate the data for Fig. 5(a).
- `PLOT/`: saccade, probability, firing-rate, raster, and LFP visualizations.
- `SPM12_UTILS/`: the subset of SPM12 utility functions used by this code.
- `UTILITIES/`: shared helpers, including the canonical output location.

Requirements

The core examples require MATLAB. The exact minimum MATLAB release has not been formalized.

Some workflows have additional requirements:

- BERT examples use the Transformer Models MATLAB package and the MATLAB toolboxes required by that package.
- Some dictionary and BERT workflows use text-processing functions such as `editDistance`.
- Figure-export sections may use export_fig.
- Tree visualization can use the MATLAB tree package.
- The optional `chatGPT/` example requires an OpenAI API key. Store it in `chatGPT/API-key.txt`; this file is ignored by Git.

Setup

Clone the repository, start MATLAB, and initialize all repository paths from the repository root:

```
repoRoot = '/path/to/HAI_LANGUAGE';
cd(repoRoot);
addpath(repoRoot);
HAI_LANGUAGE_pathsLoad(repoRoot);
```

The explicit `repoRoot` argument is recommended so setup does not depend on the MATLAB working directory.

Getting started

The simplest examples use dictionaries bundled with the repository:

```
MAIN_HAI_DICTIONARY_v0
```

`MAIN_HAI_DICTIONARY_v0.m` through `MAIN_HAI_DICTIONARY_v5.m` cover progressively larger two- and three-level language hierarchies.

Useful entry points include:

- `HAI_RUN`: run a configured hierarchical model.
- `HAI_DefaultParams` and `HAI_initialiseParams`: create and initialize model parameters.
- `HAI_disp`: inspect a hierarchical structure.
- `HAI_compare`: compare hierarchical structures.
- `HAI_getSaccades`: extract the simulated saccade count.
- `HAI_MDPtoFiringRates`, `HAI_extendFiringRates`, and `HAI_fireToSpikes`: convert model activity for neural and NeuroSequences analyses.

Paper simulations

Fig. 5(a)

Run the four control/dyslexic variants for four-letter words:

```
Sim1_CM_4l
Sim1_DM1_4l
Sim1_DM2_4l
Sim1_DM_4l
```

Then run the corresponding eight-letter simulations:

```
Sim1_CM_8l
Sim1_DM1_8l
Sim1_DM2_8l
Sim1_DM_8l
```

After all simulation outputs are present, generate Fig. 5(a):

```
sigma_parameter = 1;
Sim1_Fig5a
```

The simulations save their `.mat` results under the canonical test root described below. `Sim1_Fig5a` reads those results and writes `fig5a.eps` in the current MATLAB directory.

BERT and novel-word workflows

- `MAIN_HAI_BERT_LOOP_s01.m`: iterative reading with a BERT-generated dictionary.
- `MAIN_HAI_BERT_LOOP_s02.m`: the same loop conditioned by a language context.
- `MAIN_HAI_BERT_LOOP_s03.m` and `s04.m`: early novel-word experiments.
- `MAIN_HAI_BERT_LOOP_s05.m`: batch novel-word simulations.
- `MAIN_HAI_BERT_LOOP_s06.m`: current known/novel-word demonstration.
- `TEST_HAI_UNKNOWN.m`: reusable novel-word simulation called by the batch workflow. It saves each generated dictionary as `BERT/BERT_v1_<NEWWORD>/BERT_v1_<NEWWORD>.m`; for example, passing `ANOTHER` creates `BERT_v1_ANOTHER`.

Two generated dictionary snapshots are versioned with explicit names: `BERT_v1_BUTTER`, used by the `s03`, `s04`, and `s06` demonstrations, and `BERT_v1_ANOTHER`, generated by `TEST_HAI_UNKNOWN`. Other intermediate dictionaries and `.mat` files, including content under `DICTIONARY/BERT_DIC/`, are not versioned. Run `s01` to generate the starting BERT state before scripts that load `BERT_v1_S01/MDP_STEP001.mat`. The batch `s05` workflow also expects a generated `DICTIONARY/word_new.mat` word list.

Data and output locations

Generated MATLAB data are intentionally excluded from Git with the `*.mat` rule. Simulation and paper-script outputs should be stored outside the source repository.

The updated simulation and paper scripts resolve the canonical test root with:

```
HAI_testsRoot()
```

which returns:

```
~/TESTS/HAI_LANGUAGE
```

The scripts create experiment-specific subdirectories below this root. This keeps source code versioned independently from large or machine-specific simulation outputs.

Notes on the variational-Bayes implementation

`VB/VB_MDP.m` differs from the corresponding SPM12 `spm_MDP_VB_X.m` update in two related ways:

1. Bayesian model averaging of hidden states can update from the current time `t` to `S`, rather than always from `1` to `S`.
2. The residual-uncertainty calculation in hierarchical schemes therefore evaluates entropy at `t`, rather than at the first state.

Citation

If you use this code, please cite the published article:

```
@article{donnarumma2025integrating,
  title   = {Integrating large language models and active inference to understand eye movements in
reading and dyslexia},
  author  = {Donnarumma, Francesco and Frosolone, Mirco and Pezzulo, Giovanni},
  journal = {Physics of Life Reviews},
  volume  = {55},
  pages   = {61--78},
  year    = {2025},
  doi     = {10.1016/j.plrev.2025.08.008}
}
```

The open preprint is available as arXiv:2308.04941.

Authors and acknowledgments

- Francesco Donnarumma — francesco.donnarumma@istc.cnr.it
- Mirco Frosolone — mirco.frosolone@istc.cnr.it
- Giovanni Pezzulo — giovanni.pezzulo@istc.cnr.it

COgnition iN ActioN Laboratory (CONAN), Institute of Cognitive Sciences and Technologies, National Research Council of Italy.

License